



TRABAJO FIN DE GRADO
GRADO EN INGENIERÍA INFORMÁTICA
MENCIÓN EN INGENIERÍA DEL SOFTWARE



Diseño y desarrollo de una aplicación de gestión de transportes: ciclo de vida completo del software

Estudiante: Pablo Manzanares López

Dirección: Paula María Castro Castro

A Coruña, julio de 2026.

Agradecimientos

A mi familia, amistades y profesorado por el apoyo, la paciencia y el acompañamiento durante este trabajo.

Resumen

Este Trabajo Fin de Grado aborda el ciclo de vida completo de una aplicación web para la gestión operativa de una empresa de transportes. El alcance comprende la elicitación y especificación de requisitos, el diseño, la implementación incremental de backend y frontend, las pruebas, la integración continua y el despliegue.

TransitOps emplea una API REST desarrollada con ASP.NET Core y PostgreSQL, y una aplicación de página única basada en React y TypeScript. El desarrollo sigue una metodología iterativa organizada en ocho sprints. El primer incremento produce un esqueleto ejecutable de extremo a extremo: modelo de dominio completo, persistencia inicial de usuarios, creación controlada del primer administrador, autenticación mediante JWT, interfaz de inicio de sesión, rutas protegidas, navegación por roles, pruebas automatizadas y ejecución reproducible con Docker Compose.

La memoria se mantiene como artefacto vivo del proyecto. En esta versión se documentan con resultados reales los requisitos, las decisiones de arquitectura y el Sprint 1; los incrementos funcionales posteriores se incorporarán conforme se completen.

Abstract

This Bachelor's Thesis addresses the complete software lifecycle of a web application for the operational management of a transport company. Its scope includes requirements elicitation and specification, design, incremental backend and frontend implementation, testing, continuous integration, and deployment.

TransitOps uses an ASP.NET Core REST API with PostgreSQL and a React and TypeScript single-page application. Development follows an iterative method organised into eight sprints. The first increment delivers an executable end-to-end walking skeleton: a complete domain model, initial user persistence, controlled first-administrator creation, JWT authentication, a login interface, protected routes, role-aware navigation, automated tests, and reproducible execution with Docker Compose.

The dissertation is maintained as a living project artefact. This version records actual requirements, architecture decisions, and Sprint 1 results; later functional increments will be incorporated as they are completed.

Palabras clave:

- Gestión de transportes
- Ciclo de vida del software
- ASP.NET Core
- React
- Pruebas automatizadas
- Docker

Keywords:

- Transport management
- Software lifecycle
- ASP.NET Core
- React
- Automated testing
- Docker

Índice general

1	Introducción	1
2	Contexto y objetivos	2
2.1	Objetivo general	2
2.2	Objetivos específicos	2
3	Marco tecnológico	4
4	Metodología	5
5	Requisitos	6
6	Arquitectura	8
6.1	Modelo de datos	8
7	Diseño detallado	10
7.1	Autenticación y autorización	10
7.2	Contrato uniforme	10
7.3	Sesión en frontend	10
8	Desarrollo iterativo	11
8.1	Sprint 1: cimientos y esqueleto autenticado	11
8.2	Sprints 2 a 8	11
9	Validación	13
10	Despliegue y reproducibilidad	15
11	Resultados	16
12	Conclusiones	17

A Trazabilidad de requisitos	19
B Procedimientos de reproducción	21
B.1 Ejecución contenerizada	21
B.2 Verificación nativa	21
Lista de acrónimos	22
Glosario	23
Bibliografía	24

Índice de figuras

6.1	Arquitectura de integración de TransitOps	8
6.2	Relaciones principales del modelo de datos completo	9
8.1	Pantalla de login servida por el stack de Docker Compose	12
9.1	Ejecución de la integración continua en verde tras publicar la rama	14

Índice de cuadros

4.1	Plan incremental del proyecto	5
5.1	Requisitos funcionales	6
9.1	Pruebas automatizadas del Sprint 1	13
A.1	Matriz requisito–sprint–evidencia	19

Introducción

Las pequeñas y medianas empresas de transporte coordinan vehículos, conductores y envíos con información que a menudo queda repartida entre hojas de cálculo, mensajes y llamadas. Esta fragmentación dificulta saber qué recursos están disponibles, reconstruir el historial de un envío y detectar asignaciones incompatibles.

TransitOps plantea una aplicación web única para ese trabajo operativo. El interés del proyecto no reside únicamente en el producto final, sino en aplicar y documentar de forma disciplinada el ciclo de vida completo del software: elicitación, requisitos, diseño, implementación, verificación, despliegue y análisis de resultados.

El alcance se mantiene deliberadamente acotado para lograr una aplicación completa. Incluye usuarios con dos roles, catálogos de vehículos, conductores y clientes, envíos, asignaciones, estados, eventos y un resumen operativo. Quedan fuera la optimización de rutas, el GPS, la facturación, la aplicación móvil y el acceso de clientes externos.

Esta memoria se organiza según las fases del ciclo de vida, complementadas por un capítulo de desarrollo iterativo que conserva la evidencia de cada sprint. Así se puede estudiar tanto la solución técnica como la evolución que conduce hasta ella.

Contexto y objetivos

El trabajo se desarrolla como TFG del Grado en Ingeniería Informática, mención de Ingeniería del Software. Tras la modificación aprobada el 19 de junio de 2026, el objetivo dejó de ser una plataforma de infraestructura cloud para centrarse en el diseño y desarrollo integral de la aplicación. La implementación anterior se conserva como fuente de conocimiento, pero el producto activo se reconstruye desde cero para que cada incremento recorra realmente todas las capas.

2.1 Objetivo general

Diseñar, construir, probar y desplegar una aplicación web mantenible que permita gestionar las operaciones esenciales de una empresa de transporte y que constituya una demostración verificable del ciclo de vida completo del software.

2.2 Objetivos específicos

- Obtener y formalizar requisitos trazables desde una entrevista con la parte interesada.
- Diseñar el dominio, las interfaces y una arquitectura separada de backend y frontend.
- Implementar incrementos verticales utilizables, con autenticación y autorización desde el comienzo.
- Automatizar pruebas de comportamiento en backend y frontend.
- Garantizar ejecución local reproducible mediante contenedores e integración continua.
- Desplegar el producto completo en un entorno accesible y evaluar los resultados.

El éxito se medirá por el cumplimiento de RF-01 a RF-14, la ejecución satisfactoria de los cuatro flujos de negocio, la ausencia de secretos en el repositorio y la posibilidad de reproducir build, pruebas y despliegue siguiendo documentación explícita.

Marco tecnológico

El backend utiliza ASP.NET Core sobre .NET 10, plataforma que integra servidor HTTP, inyección de dependencias, validación, autenticación y autorización [1]. Entity Framework Core actúa como [mapeo objeto-relacional \(ORM\)](#) y mantiene migraciones versionadas [2]; PostgreSQL proporciona persistencia relacional y restricciones transaccionales [3].

El frontend es una [aplicación de página única \(SPA\)](#) creada con React [4], TypeScript y Vite. React Router resuelve navegación y rutas protegidas; Vitest y React Testing Library verifican el comportamiento desde la perspectiva de la persona usuaria.

La autenticación se basa en [JSON Web Token \(JWT\)](#), estándar definido por RFC 7519 [5]. El token firmado transporta la identidad y el rol, mientras que la API conserva la autoridad final sobre cada operación. Las contraseñas se procesan con el algoritmo adaptativo ofrecido por ASP.NET Core Identity y nunca se almacenan en claro.

Docker encapsula PostgreSQL, API y servidor web. Docker Compose describe sus relaciones y GitHub Actions ejecuta la [integración continua \(CI\)](#): restauración, compilación, migración y pruebas. Esta automatización temprana reduce la distancia entre el entorno local y el producto verificable, en línea con las prácticas de entrega continua [6].

Capítulo 4

Metodología

Se adopta un proceso iterativo e incremental. La entrevista simulada con la responsable de operaciones constituye la fuente primaria; de ella se deriva la especificación funcional, y el roadmap distribuye los requisitos en ocho incrementos verticales. Cada sprint incluye diseño detallado, implementación de API e interfaz cuando corresponde, pruebas y actualización documental.

Cuadro 4.1: Plan incremental del proyecto

Sprint	Incremento	Requisitos principales
1	Esqueleto autenticado	RF-01, RF-02, RF-13
2	Catálogos	RF-05, RF-06, RF-07
3	Envíos y filtros	RF-08, RF-12
4	Asignación y ciclo de vida	RF-09, RF-10
5	Historial de eventos	RF-11
6	Usuarios, contraseña y resumen	RF-03, RF-04, RF-14
7	Sistema completo y despliegue	RNF y E2E
8	Documentación y defensa	Cierre

El modelo completo se diseña en el primer sprint para evitar decisiones locales incompatibles. Su implementación, sin embargo, se realiza mediante migraciones incrementales, conservando una entrega pequeña y demostrable. La definición de hecho de cada sprint exige build y pruebas en verde; los cambios relevantes se registran en el contexto del repositorio.

Capítulo 5

Requisitos

Los actores son operador, administrador, público no identificado e instalador. Conductores y clientes son información del dominio, pero no usuarios del sistema. Los requisitos se formalizaron sin decisiones técnicas para separar el problema de la solución.

Cuadro 5.1: Requisitos funcionales

ID	Descripción	Prioridad
RF-01	Acceso con usuario y contraseña y permisos por rol.	Alta
RF-02	Creación controlada del primer administrador.	Alta
RF-03	Cambio de contraseña propia confirmando la actual.	Media
RF-04	Administración de usuarios, roles y activación.	Media
RF-05	Gestión y baja lógica de vehículos.	Alta
RF-06	Gestión y baja lógica de conductores.	Alta
RF-07	Gestión y baja lógica de clientes.	Media
RF-08	Creación, consulta y edición de envíos.	Alta
RF-09	Asignación no duplicada de vehículo y conductor.	Alta
RF-10	Estados planificado, en curso, entregado y cancelado.	Alta
RF-11	Historial trazable de eventos del envío.	Alta
RF-12	Visibilidad y filtros operativos de envíos.	Alta
RF-13	Validación y mensajes comprensibles.	Alta

Las dieciséis reglas de negocio concretan asignaciones, recursos activos, capacidad, fechas, transiciones, autoría de eventos, administración y bajas lógicas. Los seis requisitos no funcionales cubren facilidad de uso, acceso seguro, fiabilidad, trazabilidad, disponibilidad y rendimiento adecuado para una empresa con unas decenas de recursos. La especificación canónica y su origen se conservan en `docs/Requirements.md` y `docs/ClientRequirements.md`.

Arquitectura

La solución separa una SPA, una [interfaz de programación de aplicaciones \(API\)](#) REST y PostgreSQL. El navegador nunca accede directamente a persistencia: todas las reglas y autorizaciones se evalúan en la API.

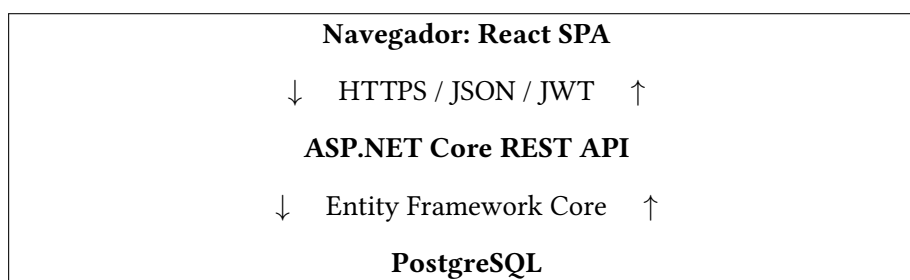


Figura 6.1: Arquitectura de integración de TransitOps

6.1 Modelo de datos

El modelo completo se diseñó antes de implementar las rebanadas funcionales. `Shipment` referencia opcionalmente cliente, vehículo y conductor; `ShipmentEvent` referencia obligatoriamente el envío y el usuario autor. Los catálogos y usuarios usan [baja lógica](#); las relaciones históricas no se borran en cascada.

El diseño detallado, con todos los atributos y restricciones, está versionado en `docs/design/DataModel.md`. La migración inicial materializa únicamente `app_users`; las demás tablas se añadirán al implementar sus requisitos.

Customer	0..1 ← *	Shipment
Vehicle	0..1 ← *	Shipment
Driver	0..1 ← *	Shipment
Shipment	1 ← *	ShipmentEvent
AppUser	1 ← *	ShipmentEvent

AppUser, Vehicle, Driver y Customer conservan estado activo; Shipment contiene estado, fechas y carga estimada.

Figura 6.2: Relaciones principales del modelo de datos completo

Diseño detallado

7.1 Autenticación y autorización

El arranque recibe un secreto de instalación mediante la cabecera `X-Bootstrap-Token`. Antes de crear el usuario comprueba que no existe un administrador activo y almacena un hash de la contraseña. El login rechaza de forma indistinguible usuario inexistente, contraseña incorrecta y usuario inactivo para no revelar información de cuentas.

Un login válido emite un JWT firmado con identificador, nombre, correo, rol y caducidad. Las políticas *operational* y *admin* se aplican en el servidor. La SPA usa el rol devuelto para adaptar la navegación, pero esa adaptación visual no sustituye a la autorización de API.

7.2 Contrato uniforme

Una respuesta correcta contiene `data` y `requestId`. Un error contiene `error.code`, `error.message`, detalles opcionales y el mismo identificador. Middleware y eventos de autenticación garantizan esta forma para 400, 401, 403, 409 y 500. La interfaz traduce el mensaje a un aviso accesible con rol `alert`.

7.3 Sesión en frontend

La sesión se conserva en `localStorage` y se descarta al expirar. Es una decisión pragmática para el Sprint 1, documentada con su riesgo frente a XSS. El endurecimiento del Sprint 7 evaluará `cookie HttpOnly` y una política de seguridad de contenidos. Los diseños de catálogos, envíos y operaciones se incorporarán en sus respectivos sprints.

Desarrollo iterativo

8.1 Sprint 1: cimientos y esqueleto autenticado

El incremento comenzó con una solución .NET y dos proyectos independientes, más una SPA TypeScript. En backend se implementaron la entidad de usuario, su configuración EF Core y la migración `InitialCreate`; el servicio de autenticación; los controladores de salud y sesión; JWT, CORS, correlación y tratamiento uniforme de excepciones.

En frontend se construyeron el cliente HTTP, un contexto de sesión persistente, la pantalla de login, el componente de error, la ruta protegida y un layout adaptable. Un administrador ve la futura entrada de usuarios, mientras que un operador recibe solo la navegación común. El resultado atraviesa navegador, proxy, API y PostgreSQL: constituye un [esqueleto andante](#) real.

Docker Compose orquesta base de datos, API y web; la API aplica migraciones al arrancar. GitHub Actions compila y prueba ambas aplicaciones y ejecuta la migración contra PostgreSQL. Esta infraestructura se mantendrá en todos los incrementos siguientes.

8.2 Sprints 2 a 8

Sprint 2 — Catálogos

Pendiente de completar vehículos, conductores y clientes.

Sprint 3 — Envíos y filtros

Pendiente de completar el CRUD y la visibilidad operativa.

Sprint 4 — Asignación y ciclo de vida

Pendiente de completar reglas de recursos, capacidad y estados.

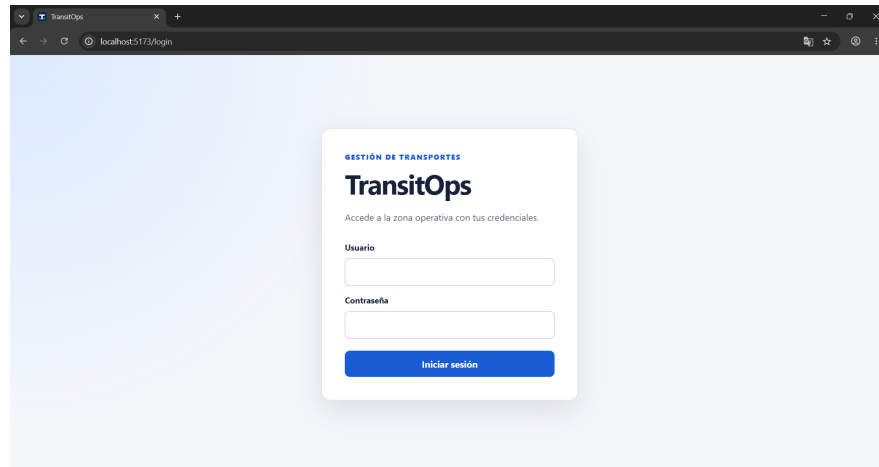


Figura 8.1: Pantalla de login servida por el stack de Docker Compose

Sprint 5 — Eventos

Pendiente de completar la trazabilidad de sucesos e incidencias.

Sprint 6 — Administración y resumen

Pendiente de completar usuarios, contraseña propia y estadísticas.

Sprint 7 — Endurecimiento y despliegue

Pendiente de pruebas de sistema y entorno accesible.

Sprint 8 — Cierre

Pendiente de consolidación de memoria, defensa y conclusiones.

Capítulo 9

Validación

La estrategia combina pruebas de integración de API y pruebas de componentes/rutas de frontend. Las primeras arrancan la aplicación completa con `WebApplicationFactory` y sustituyen solo PostgreSQL por una base en memoria aislada. Las segundas simulan la API y manipulan la interfaz como lo haría una persona usuaria.

Cuadro 9.1: Pruebas automatizadas del Sprint 1

Comportamiento	Capa	Resultado
Primer administrador y hash de contraseña	Backend	Correcto
Bloqueo de un segundo bootstrap	Backend	Correcto
Login válido, erróneo y usuario inactivo	Backend	Correcto
Rutas protegidas 401 y 403	Backend	Correcto
Contrato de validación	Backend	Correcto
Redirección sin sesión	Frontend	Correcto
Login, persistencia y error visible	Frontend	Correcto
Navegación adaptada al rol administrador	Frontend	Correcto

La ejecución local al cierre del incremento produjo dieciséis pruebas backend, organizadas por servicio y controlador, y cuatro frontend correctas, además de build de producción de ambos proyectos. La CI repite las mismas comprobaciones y valida la migración sobre PostgreSQL 16. Las pruebas funcionales de sistema crecerán con cada flujo y se consolidarán en el Sprint 7.

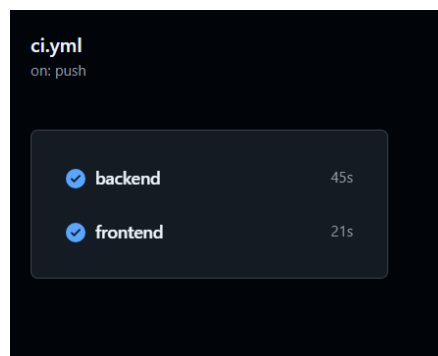


Figura 9.1: Ejecución de la integración continua en verde tras publicar la rama

Despliegue y reproducibilidad

Desde el primer sprint, `docker compose up --build` crea tres servicios. PostgreSQL mantiene un volumen persistente y expone una comprobación de salud; la API espera a que la base esté disponible, aplica migraciones y escucha en el puerto 8080; Nginx sirve la SPA en el puerto 5173 y reenvía `/api` al backend.

Los valores del Compose permiten una demostración local inmediata. El fichero `.env.example` enumera las variables que deben sustituirse y `.env` queda excluido del control de versiones. En particular, claves JWT, token de bootstrap y contraseña de base de datos deberán gestionarse como secretos en cualquier entorno accesible.

El objetivo de este sprint es reproducibilidad local, no operación pública. La selección del destino accesible, HTTPS, copias de seguridad, observabilidad mínima y automatización de despliegue pertenece al Sprint 7.

Resultados

El Sprint 1 entrega el primer resultado demostrable: desde un navegador se puede introducir un usuario y contraseña, autenticar contra la API real y acceder a una zona que no existe para visitantes anónimos. El rol aparece de forma comprensible y altera la navegación. La creación del primer administrador queda separada del uso diario y se bloquea una vez cumplida su función.

El incremento también reduce riesgo futuro. El modelo de datos completo hace visibles las relaciones y restricciones antes de construir catálogos; el contrato de error evita tratamientos distintos por módulo; la migración, los contenedores y la CI establecen un camino repetible que los sprints posteriores ampliarán.

Todavía no procede evaluar la utilidad operacional completa, pues no se han implementado vehículos, conductores, clientes ni envíos. Esa evaluación se actualizará con métricas de cobertura funcional, defectos y resultados de los cuatro flujos al final del Sprint 7.

Conclusiones

La primera iteración confirma la viabilidad de la arquitectura y del enfoque metodológico. Se ha pasado de requisitos de negocio a un incremento vertical verificable sin incorporar aún complejidad funcional innecesaria. La separación SPA/API, las políticas de autorización, el modelo versionado y las pruebas automatizadas proporcionan una base mantenible.

La principal lección temprana es que un esqueleto transversal descubre problemas de integración antes de que el dominio crezca: configuración EF de pruebas, forma de errores de autenticación, persistencia de sesión y proxy web se han validado ya. También se ha hecho explícita la deuda aceptada de `localStorage`, que se revisará durante el endurecimiento.

Estas conclusiones son provisionales. Se completarán tras implementar el alcance funcional, desplegar la aplicación y analizar las pruebas de sistema. Las líneas futuras fuera del TFG podrán incluir rutas, GPS, facturación o acceso de clientes, siempre después de validar la utilidad del núcleo operativo.

Apéndice

Trazabilidad de requisitos

Cuadro A.1: Matriz requisito-sprint-evidencia

Requisito	Sprint	Evidencia
RF-01	S1	Login API y SPA; pruebas de credenciales, inactividad y rutas 401/403.
RF-02	S1	Endpoint protegido por token; hash; conflicto con administrador activo.
RF-13	S1-S7	Contrato común, validación de modelo y componente de alerta.
RF-05-07	S2	Pendiente.
RF-08, RF-12	S3	Pendiente.
RF-09, RF-10	S4	Pendiente.
RF-11	S5	Pendiente.
RF-03, RF-04, RF-14	S6	Pendiente.

APÉNDICE A. TRAZABILIDAD DE REQUISITOS

RNF-01– 06	S1–S7	Evidencia incremental; validación final pendiente en S7.
---------------	-------	--

Procedimientos de reproducción

B.1 Ejecución contenerizada

Con Docker Desktop disponible, desde la raíz se ejecuta:

```
copy .env.example .env
docker compose up --build
```

La interfaz queda en <http://localhost:5173> y la salud de API en <http://localhost:8080/api/v1/health>. El primer administrador se crea una sola vez mediante POST `/api/v1/auth/bootstrap-admin`, enviando JSON con usuario, correo y contraseña y la cabecera X-Bootstrap-Token. El fichero `.env` es obligatorio, queda ignorado por Git y se crea desde `.env.example`; sus valores son exclusivamente locales.

B.2 Verificación nativa

```
dotnet restore TransitOps.slnx
dotnet build TransitOps.slnx
dotnet test TransitOps.slnx
cd frontend
npm ci
npm run build
npm run test
```

La migración se valida con `dotnet tool restore` y la opción `--migrate-only` de la API contra una instancia PostgreSQL configurada.

Lista de acrónimos

API interfaz de programación de aplicaciones. [8](#)

CI integración continua. [4](#)

JWT JSON Web Token. [4](#)

ORM mapeo objeto-relacional. [4](#)

SPA aplicación de página única. [4](#)

Glosario

baja lógica desactivación de un registro sin eliminar sus datos ni sus relaciones históricas. [8](#)

esqueleto andante incremento mínimo que atraviesa todas las capas de una aplicación y demuestra su integración. [11](#)

Bibliografía

- [1] Microsoft. (2026) Asp.net core documentation. [Online]. Available: <https://learn.microsoft.com/aspnet/core/>
- [2] ——. (2026) Entity framework core documentation. [Online]. Available: <https://learn.microsoft.com/ef/core/>
- [3] PostgreSQL Global Development Group. (2026) Postgresql documentation. [Online]. Available: <https://www.postgresql.org/docs/>
- [4] Meta Open Source. (2026) React documentation. [Online]. Available: <https://react.dev/>
- [5] M. Jones, J. Bradley, and N. Sakimura, “Json web token (jwt),” IETF, Tech. Rep. RFC 7519, 2015.
- [6] J. Humble and D. Farley, *Continuous Delivery*. Addison-Wesley, 2010.